

Travail Pratique : Conception et Développement d'une API RESTful

Professeur : Yasser Lamalem

1. Modélisation des Données

L'API doit interagir avec une base de données relationnelle composée de deux entités principales. Vous devez générer le schéma physique ou configurer votre ORM en respectant les spécifications suivantes :

1.1. Entité *Categorie*

- **id** : Clé primaire, auto-incrémentée.
- **title** : Chaîne de caractères (obligatoire, unique).
- **description** : Texte long (optionnel).
- *Traçabilité* : Champs pour la date de création (**created_at**) et la date de dernière modification (**updated_at**).

1.2. Entité *Produit*

- **id** : Clé primaire, auto-incrémentée.
- **title** : Chaîne de caractères (obligatoire).
- **description** : Texte long (obligatoire).
- **price** : Décimal (obligatoire, format : 10 chiffres au total dont 2 après la virgule).
- **image** : Chaîne de caractères (optionnel, URL ou chemin de l'image).
- **category_id** : Clé étrangère pointant vers l'entité *Categorie*. Une contrainte d'intégrité référentielle stricte (**ON DELETE CASCADE**) doit être appliquée.
- *Traçabilité* : Champs pour la date de création (**created_at**) et la date de dernière modification (**updated_at**).

2. Spécifications de l'API (Endpoints)

Le backend doit exposer les points de terminaison suivants au format JSON.

2.1. GET /products

Description : Récupérer la liste des produits.

Code HTTP attendu : 200 OK.

Bonus technique : Implémenter la pagination et un filtre par **category_id**.

2.2. GET /products/{id}

Description : Récupérer les détails d'un produit.

Codes HTTP attendus :

- 200 OK si le produit est trouvé.
- 404 Not Found si le produit n'existe pas.

2.3. POST /products

Description : Créer un nouveau produit. Accepte un objet JSON dans le corps de la requête (*Body*).

Codes HTTP attendus :

- 201 Created en cas de succès, retournant l'objet fraîchement créé.
- 422 Unprocessable Entity (ou 400 Bad Request) si les données fournies sont invalides.

3. Contraintes Techniques et Architecture

Peu importe le langage ou le framework choisi, le projet doit se conformer aux exigences architecturales suivantes :

3.1. Séparation des couches

Le code ne doit pas être concentré uniquement dans les contrôleurs. Utilisez des classes de *Services* pour la logique métier et des *Repositories* (ou DAO) pour l'interaction avec la base de données. L'utilisation de DTO (*Data Transfer Objects*) est exigée pour le transfert des données.

3.2. Gestion des Erreurs et Validation

Vous devez implémenter un mécanisme de validation côté serveur pour les requêtes entrantes. En cas d'erreur de validation (ex : champs manquants, format incorrect), l'API doit **impérativement** retourner la structure JSON suivante :

Format de réponse attendu en cas d'erreur

```
1 {
2   "message": "The title field is required. (and 3 more errors)",
3   "errors": {
4     "title": [
5       "The title field is required."
6     ],
7     "price": [
8       "The price field must be greater than 0."
9     ],
10    "description": [
11      "The description field is required."
12    ],
13    "image": [
14      "The image field is required."
15    ]
16  }
17 }
```

Mettez en place un gestionnaire d'exceptions global (*Global Exception Handler*) pour intercepter ces erreurs proprement, sans jamais exposer la *stacktrace* au client.

3.3. Sécurité

Veillez à vous prémunir contre les failles d'injection SQL en utilisant systématiquement des requêtes préparées ou les mécanismes sécurisés de votre ORM.